

vasic-digital

Vasic Digital

Héros

L'ingénierie logicielle native pour les AI, conçue pour inspirer confiance.

N'importe qui peut brancher une application à un LLM en une après-midi. La partie difficile — celle qui détermine si un système AI n'est qu'une démo ou un produit fiable — réside dans tout ce qui entoure le modèle : l'abstraction des fournisseurs capable de résister à une panne, l'orchestration qui maintient les agents sur leur tâche, la vérification qui détecte les réponses fantaisistes d'un modèle, et la gouvernance qui prouve que l'ensemble fonctionne comme prévu. C'est cette partie complexe que construit Vasic Digital. Nous concevons et livrons des systèmes de développement AI — modèles, agents, orchestration et infrastructure — qui transforment les grands modèles de langage en logiciels fiables, accompagnés d'une couche de gouvernance pour en garantir l'intégrité. Tout repose sur une règle intransigeante : une fonctionnalité n'est pas « terminée » quand les tests passent ; elle l'est quand un utilisateur réel peut s'en servir, et qu'il existe des preuves tangibles pour le démontrer.

À propos

Vasic Digital est une pratique d'ingénierie ciblée qui développe une famille interconnectée de produits et de modules réutilisables pour le développement AI. Plutôt qu'un monolithe, le travail s'organise comme une flotte : des applications produits majeures s'appuyant sur des dizaines de petits modules indépendants, testés séparément et découplés — de sorte que les composants éprouvés soient réutilisés dans chaque produit au lieu d'être réinventés. Le langage principal est **Go**, complété par **Kotlin / Kotlin Multiplatform, TypeScript/React, Python, Swift** et **Shell**, choisis en fonction des besoins : Go pour les services et bibliothèques à haut débit, Kotlin pour les outils de

provisionnement et les applications mobiles multiplateformes, TypeScript pour les interfaces typées, Python pour l'intégration AI/ML.

Ce qui unit cette flotte, c'est une discipline rendue mécanique plutôt qu'aspirationnelle. Chaque projet hérite d'un **Constitution** d'ingénierie partagé sous forme de sous-module Git — ainsi, une règle renforcée une fois se propage à travers une flotte de plus de 140 dépôts — et chaque fonctionnalité annoncée par un produit doit être validée par un test automatisé produisant des preuves avant d'être considérée comme livrée. Il ne s'agit pas d'un discours marketing plaqué sur le travail ; c'est le modèle opérationnel qui le sous-tend. L'avantage réel réside dans l'effet cumulatif : comme les préoccupations génériques résident dans des modules découplés et testés indépendamment, une correction ou une amélioration apportée en un seul endroit bénéficie instantanément à tous les produits, et chaque nouveau système est assemblé à partir de composants dont la fiabilité a déjà été éprouvée.

Ce que nous faisons

Développement basé sur les AI. Nous construisons le socle des systèmes AI de bout en bout :

- **Accès multi-fournisseurs aux LLM** — une abstraction propriétaire couvrant plus de 40 fournisseurs (Anthropic/ Claude, OpenAI, DeepSeek, Gemini, Mistral, Cohere, Groq, xAI/Grok, Qwen, Perplexity, OpenRouter, Together AI, Replicate, Cerebras, Cloudflare Workers AI, SiliconFlow, et un retour local sur Ollama) derrière une seule interface, avec réessais, disjoncteurs et vérifications d'état.
- **Orchestration d'agents** — plans de contrôle d'agents de codage CLI en mode headless, workflows agentiques basés sur des graphes, consensus multi-tours par « débat AI », et moteurs d'exécution DAG/pipeline.
- **Vérification des LLM** — une couche de confiance qui évalue les modèles avec un filtre de compréhension obligatoire (« Vois-tu mon code ? »), ainsi que des tests de latence, de streaming, d'appel de fonctions, de vision et de embeddings, exportant une configuration validée uniquement.
- **Récupération et mémoire** — RAG, bases de données vector, embeddings, et moteurs de mémoire fusionnée pour agents (Mem0 + Cognee + Letta) avec compression de contexte illimité.
- **LLM défensifs** — garde-fous, détection de PII, scénarios adverses de red team, et canonisation des entrées.

La famille de produits Helix. Notre gamme phare couvre l'intégralité du cycle de développement AI :

- **HelixTrack** — une alternative libre à JIRA (le produit phare de la gamme Helix-Track).
- **HelixAgent** — un service LLM collaboratif permettant à plusieurs modèles de débattre avant de livrer une réponse consensuelle.
- **HelixCode** — une plateforme de développement AI distribuée, répartissant les tâches entre des workers gérés par SSH avec points de contrôle et retour arrière.
- **HelixLLM** — un binaire unique, six modes : inférence compatible OpenAI et Anthropic, du portable au cluster, via HTTP/3.
- **HelixCluster** — un système d'exploitation distribué pour le calcul AI, des GPU de datacenter aux appareils edge portables.
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — l'abstraction fournisseur, le plan de contrôle des agents et la source de vérité pour la vérification.
- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — mémoire, compétences régies, développement piloté par spécifications, construction d'applications, traduction vérifiée, terminaux à confiance zéro, Git fédéré, mises à jour sécurisées de OTA et cloud gaming auto-hébergé.

Outils et utilitaires (vasic-digital utils). Des outils de qualité professionnelle, autonomes : **Catalogizer** (gestion multi-protocole de collections de médias chiffrés), **Courses-Creator** (production de cours AI au format vidéo à partir de Markdown), **VisionEngine** (vision par ordinateur + perception d'interface LLM), **DocProcessor** (cartographie des fonctionnalités à partir de la documentation pour l'assurance qualité), **Docs Chain** (synchronisation bidirectionnelle de documents et bases de données avec hachage de contenu), **Herald** (notifications multicanaux en langage naturel), **task_bridge** (synchronisation bidirectionnelle entre tâches et tableaux), et la **Suite de modules réutilisables Vasic Digital** — la « bibliothèque standard » `digital.vasic.*` des modules d'infrastructure, des primitives AI et des garde-fous.

Automatisation de l'infrastructure (Server Factory). Notre héritage DevOps : **Mail Server Factory** et le **Cadre de base Server Factory**, qui transforment des configurations JSON déclaratives en serveurs entièrement provisionnés et conteneurisés sous Docker, sur divers types de connexions et distributions Linux, ainsi que des outils de création d'images VM (Qemu-Utills, Parallels-Utills) et des usines de services associés.

Technologies

Fondées sur notre pile technologique réelle :

- **Langages** : Go (dominant), Kotlin & Kotlin Multiplatform, TypeScript, Python, Swift, Shell, avec des spécifications formelles PL/pgSQL et même TLA+ pour les travaux sur les systèmes distribués.
- **AI / LLM** : accès multi-fournisseurs (43+ adaptateurs), Model Context Protocol (MCP), RAG, bases de données vector et embeddings, algorithmes de planification (HiPlan, MCTS, Tree of Thoughts), LLMops, outils d'évaluation (SWE-bench/ HumanEval/MMLU) et TTS (Bark, SpeechT5).
- **Backend** : Gin (Go), gRPC + Protocol Buffers, HTTP/3 (QUIC), WebSockets, interfaces Angular et React, messagerie Kafka/ RabbitMQ.
- **Données** : PostgreSQL, SQLite, SQLCipher (chiffrées au repos), Redis, Neo4j, ClickHouse et stockage d'objets (MinIO/ S3/GCS/Azure).
- **Infrastructure / DevOps** : Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/ Parallels, et CI/CD via GitHub Actions, Gradle et Make.
- **Tests / Assurance qualité** : le framework anti-biais HelixQA, des harnais de tests par module avec portes de mutation, go test -race, outils de régression visuelle, tests sur appareils ADB, portes SonarQube et analyses de sécurité (semgrep, gosec, trivy, snyk, gitleaks, nancy).

Qualité et gouvernance — notre différenciation

Deux piliers assurent la cohérence et la fiabilité de l'ensemble de la flotte :

- **HelixConstitution** — un recueil de règles d'ingénierie universel, indépendant des projets, intégré sous forme de sous-module Git et hérité par chaque projet au sein d'une flotte de plus de 140 dépôts. Il formalise une discipline incontournable — des garde-fous anti-bidonnage, une immunité contre les faux positifs, la sécurité des données et des hôtes, ainsi que des règles de documentation et de couverture — qu'un projet peut étendre, mais jamais affaiblir. Une simple mise à jour du sous-module actualise les règles partout ; des mécanismes de propagation vérifient littéralement la présence des clauses requises dans toute la flotte, et chaque garde-fou est accompagné d'un test de mutation prouvant qu'il n'est pas une coquille vide. La gouvernance devient un fait vérifiable, et non plus un vœu pieux.

- **HelixQA** — une orchestration QA anti-bidonnage. Elle exécute des batteries de tests YAML rédigés ainsi que des sessions QA entièrement autonomes, combinant LLM et vision par ordinateur, sur Android, Android TV, Web et Desktop. Elle refuse de valider un PASS sans preuves d'exécution capturées (captures d'écran, logcat, vidéo, traces de pile). « Nous l'avons testé » devient « voici la vidéo, le logcat et le ticket ».

Énoncé de positionnement

N'importe qui peut connecter une application à un LLM. **Vasic Digital** construit ce qui est difficile : des systèmes **AI** vérifiables, réutilisables et honnêtes — un socle **AI** agnostique aux fournisseurs, un cycle de vie de produits **Helix** par-dessus, et une discipline constitutionnelle assortie de preuves garantissant que ce qui est livré fonctionne réellement. Nous ne vous demandons pas de croire sur parole la coche verte. Nous vous montrons les preuves qui la sous-tendent.

Contact

Construisons quelque chose de vérifiable.

- **Email** : i@mvasic.ru
- **GitHub** : github.com/vasic-digital